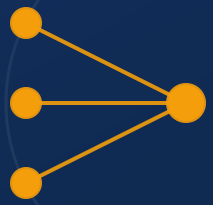
GLOBAL SOLUTION · APRENDIZADO PROFUNDO

Segurança pública inteligente em São Paulo

Classificação de ocorrências como **roubo** ou **furto** com uma rede neural MLP



Objetivo. Construir, treinar e comparar três arquiteturas de MLP sobre dados públicos da SSP-SP e disponibilizar o modelo escolhido em uma aplicação web para testar novas ocorrências.

O modelo classifica o **tipo de uma ocorrência já registrada**. Ele **não prevê** se, onde ou quando um crime vai acontecer.

DISCIPLINA

Aprendizado Profundo com Redes Neurais Convolucionais

INTEGRANTES

Gabriel Guimarães de Oliveira · RM567835
Pedro Paulo Ferreira Agnelo D'angelo · RM567564
Mark Lima Leal · RM566760

APLICAÇÃO WEB · REPOSITÓRIO

spmobilecrimeclassifier.streamlit.app
github.com/ZuziR6/sp-mobile-crime-classifier

Dados públicos da SSP-SP: 158.859 boletins de São Paulo em 2025

PROBLEMA

Distinguir **roubo** de **furto** a partir das características de uma ocorrência registrada na capital paulista (dados tabulares).

RECORTE E QUANTIDADE DE REGISTROS

383.635 registros na planilha SSP-SP (Celulares Subtraídos 2025)

213.411 fatos em São Paulo (campo CIDADE) ocorridos em 2025

181.707 natureza Furto (art. 155) ou Roubo (art. 157)

158.859 boletins únicos, sem ambíguos nem inconsistentes

Fonte pública: Secretaria da Segurança Pública (SSP-SP), Portal de Estatísticas — base *Celulares Subtraídos* 2025.
Link: ssp.sp.gov.br/estatistica/consultas

Local do fato, não do registro: o recorte usa `CIDADE` (“cidade dos fatos”), não o município da delegacia que registrou o boletim.

ATRIBUTOS SELECIONADOS E JUSTIFICATIVA

ATRIBUTO	POR QUE USAR
Bairro	O perfil urbano varia por região (centro, terminais, áreas residenciais)
Mês	Possível sazonalidade
Dia da semana	Dia útil e fim de semana têm dinâmicas distintas
Período do dia	Manhã, tarde, noite ou madrugada
Tipo de local	Via pública, terminal, residência, comércio...
Subtipo de local	Detalha o tipo (transeunte, interior de ônibus...)

Rótulo: Furto = 0 · Roubo = 1, a partir da natureza (`RUBRICA`), que **não entra** nas entradas (evita *data leakage*).
Fora do modelo: campos que descrevem a dinâmica do fato, identificação do boletim, endereço exato e coordenadas.

Seis atributos categóricos viram 293 entradas binárias

VARIÁVEL	CATEGORIAS	TRATAMENTO
BAIRRO	1.821	Ausente → `NAO_INFORMADO`; raras agrupadas
MES	12	Extraído da data da ocorrência; categórico
DIA_SEMANA	7	Extraído da data da ocorrência; categórico
DESCR_PERIODO	6	48,6% sem período → `NAO_INFORMADO`
DESCR_TIPOLOCAL	28	Texto limpo (`strip`)
DESCR_SUBTIPOLOCAL	197	Texto limpo (`strip`)

Antes das variáveis: removidos 44 boletins com roubo e furto ao mesmo tempo e 820 com atributos divergentes entre linhas; uma linha por boletim.

CODIFICAÇÃO

One-hot (`OneHotEncoder`), com categorias raras (< 50 ocorrências) agrupadas em "infrequente". Resultado: **293 entradas**.

ESCALONAMENTO

Não necessário: todas as entradas são binárias (0/1) e já estão na mesma escala.

SEM VAZAMENTO

O codificador é ajustado **somente no treino**; validação e teste apenas são transformados.

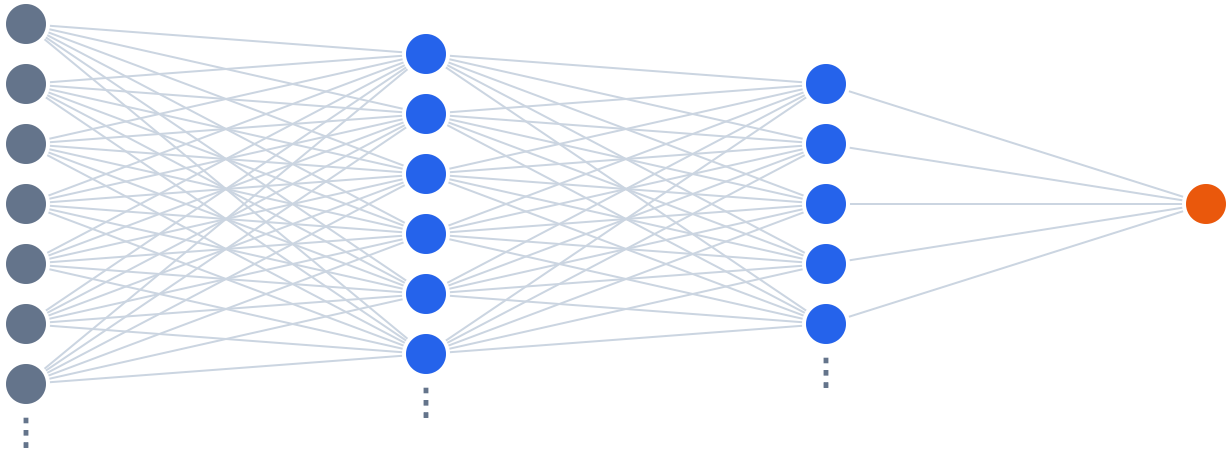
SEPARAÇÃO DOS CONJUNTOS · ESTRATIFICADA, `RANDOM_STATE=42`



Validação: 23.829 Teste: 23.829

Proporção de classes preservada nos três conjuntos: **63,31% furto** (100.571) e **36,69% roubo** (58.288). O teste só é usado na avaliação final.

MLP: 293 entradas → 64 → 32 → 1 saída sigmoide



Arquitetura escolhida: **MLP_1_Compacta** · 293 → 64 → 32 → 1 · 20.929 parâmetros treináveis (18.816 + 2.080 + 33)

COMO A REDE FUNCIONA

- Cada camada oculta calcula `ReLU(W·x + b)`: combina as entradas e mantém só os sinais positivos.
- O **Dropout** desliga 20% dos neurônios a cada passo do treino, reduzindo o overfitting.
- A saída **sigmoide** gera um número entre 0 e 1: a probabilidade de **roubo**.

DECISÃO

Probabilidade ≥ **0,50** → **roubo**; caso contrário → **furto**. A probabilidade de furto é 1 − P(roubo).

ENTRADA

293 neurônios, um por categoria codificada (bairro, mês, dia, período, tipo e subtipo de local).

CAMADAS OCULTAS

Densas, com **ReLU** e regularização **L2 (1e-4)**, seguidas de Dropout 0,20.

SAÍDA

1 neurônio com **sigmoid**: adequado a classificação binária e compatível com *binary crossentropy*.

Mesma configuração nas três redes: só a capacidade muda

FUNÇÃO DE PERDA

Binary crossentropy

Natural para saída sigmoide e rótulos 0/1; penaliza previsões confiantes e erradas.

OTIMIZADOR

Adam

Adapta o passo por parâmetro; converge bem sem ajuste fino.

TAXA DE APRENDIZADO

1×10^{-3}

Valor inicial; reduzida pela metade quando a validação estagna (mínimo 1e-5).

BATCH SIZE

256

435 passos por época com 111.201 amostras de treino.

ÉPOCAS

até 50

Limite máximo; o early stopping interrompeu em 19, 16 e 11 épocas.

REGULARIZAÇÃO

Dropout 0,20 + L2

L2 de 1e-4 nos pesos das camadas ocultas, contra overfitting.

EARLY STOPPING

paciência 7

Monitora `val_loss` e restaura os pesos da melhor época.

REPRODUTIBILIDADE

semente fixa

Semente 42 (+ índice do modelo) e divisão estratificada.

EXECUÇÃO OBSERVADA DO TREINAMENTO

MODELO	ÉPOCAS EXECUTADAS	MELHOR ÉPOCA (VAL_LOSS)	MENOR VAL_LOSS	REDUÇÃO DA TAXA DE APRENDIZADO
MLP_1_Compacta	19	12	0,3960	3× (épocas 11, 15 e 18)
MLP_2_Intermediaria	16	9	0,4057	3× (épocas 8, 12 e 15)
MLP_3_Profunda	11	4	0,4035	2× (épocas 7 e 10)

Três arquiteturas testadas: mais capacidade não trouxe ganho

ARQUITETURA	CAMADAS OCULTAS	PARÂMETROS	O QUE MUDOU E POR QUÊ	EFEITO NA VALIDAÇÃO
MLP_1_Compacta	64 → 32	20.929	Linha de base simples	Acurácia 0,8302 · val_loss **0,3960** (melhor época 12)
MLP_2_Intermediaria	128 → 64 → 32	48.001	+1 camada e o dobro de neurônios na primeira: testa se mais profundidade ajuda	Acurácia 0,8296 · val_loss 0,4057 (melhor época 9); sem ganho
MLP_3_Profunda	256 → 128 → 64	116.481	Quatro vezes os neurônios da primeira camada: testa maior capacidade	Acurácia 0,8302 · val_loss 0,4035 (melhor época 4); overfitting precoce

Fixos nas três: ReLU, Dropout 0,20, L2 1e-4, Adam (1e-3), batch 256 e callbacks. Assim, a diferença de resultado vem da capacidade da rede.

EXPERIMENTOS COMPLEMENTARES NA MLP_1 (VALIDAÇÃO)

LIMIAR 0,45

F1 macro 0,8210 (vs 0,8185). Troca **−1,1 p.p. de precision** por **+2,5 p.p. de recall** em roubo; ganho marginal.

PESOS DE CLASSE

Recall de roubo sobe a 0,8512, mas a precision cai a 0,7184; F1 macro **0,8157**, sem melhora.

L2 = 0,001

Praticamente igual: acurácia 0,8301 e F1 macro **0,8189** (+0,0004).

Ressalva: os experimentos com pesos de classe e L2 partiram da taxa de aprendizado já reduzida ($1{,}25 \times 10^{-4}$); são indicativos e não alteram a comparação das três arquiteturas.

07 · COMPARAÇÃO E ESCOLHA DO MODELO

MLP_1_Compacta: melhor equilíbrio entre desempenho e simplicidade

CONJUNTO DE VALIDAÇÃO · LIMIAR 0,50 · PRECISION, RECALL E F1 DA CLASSE ROUBO

MODELO	ACURÁCIA	PRECISION	RECALL	F1	F1 MACRO	VAL_LOSS
MLP_1_Compacta ✓	**0,8302**	**0,7602**	**0,7851**	**0,7724**	**0,8185**	**0,3960**
MLP_3_Profunda	0,8302	0,7647	0,7761	0,7703	0,8178	0,4035
MLP_2_Intermediaria	0,8296	0,7640	0,7750	0,7695	0,8172	0,4057

As diferenças entre os modelos são mínimas (até 0,0013 de F1 macro): na prática, há empate técnico.

MODELO ESCOLHIDO

MLP_1_Compacta

- Menor perda de validação (0,3960)
- Maior F1 macro (0,8185) e acurácia igual à melhor
- Menor capacidade: 20.929 parâmetros, cerca de 5,6× menos que a MLP_3
- Menor sinal de overfitting

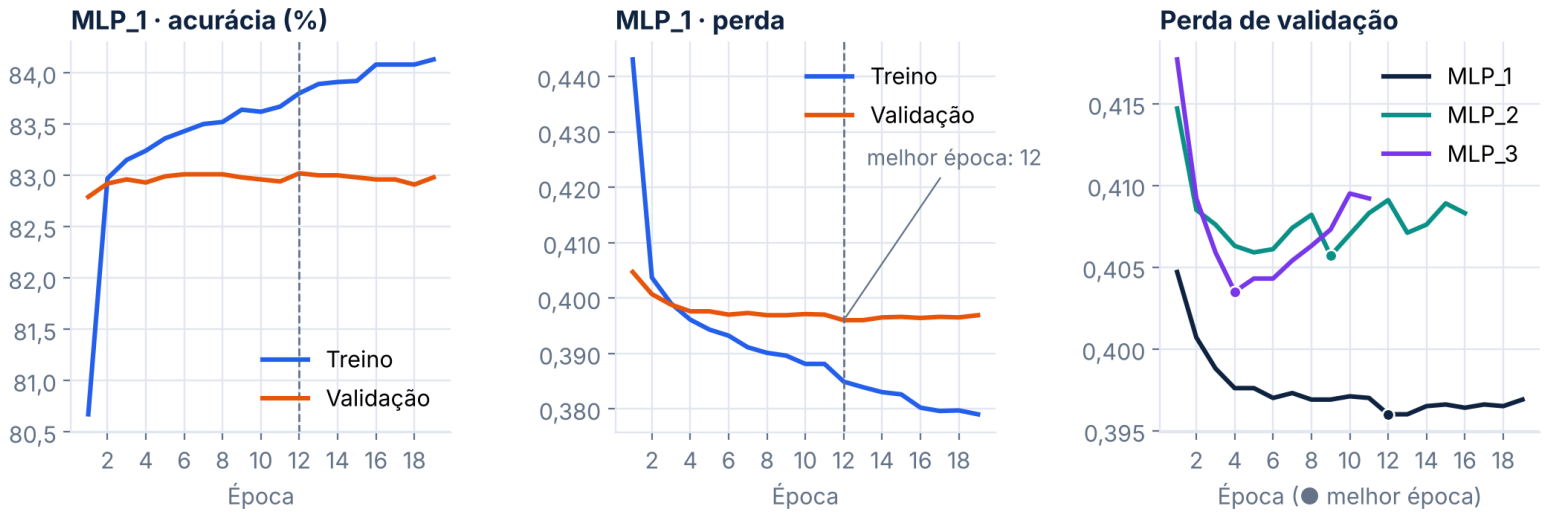
Escolha feita só com a validação. Com desempenho equivalente, prevaleceu a **parcimônia**.

AVALIAÇÃO FINAL · CONJUNTO DE TESTE (USADO UMA ÚNICA VEZ) · LIMIAR 0,50

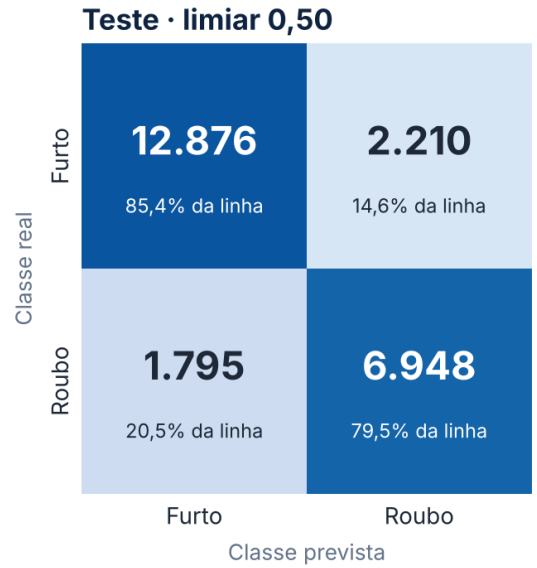
83,19%	75,87%	79,47%	77,63%	82,08%
Acurácia	Precision (roubo)	Recall (roubo)	F1 (roubo)	F1 macro

Validação e teste são muito próximos (acurácia 83,02% e 83,19%), o que indica boa generalização.

Generalização estável, com overfitting leve controlado



Treino × validação da MLP_1 (linha tracejada = melhor época) e perda de validação das três arquiteturas.



OVERFITTING

Leve na MLP_1: a perda de validação estabiliza na época 12 enquanto a de treino segue caindo (gap de 1,15 p.p. na acurácia). Quanto maior a rede, mais cedo a validação piora (MLP_3: época 4). O early stopping restaura os melhores pesos.

UNDERFITTING

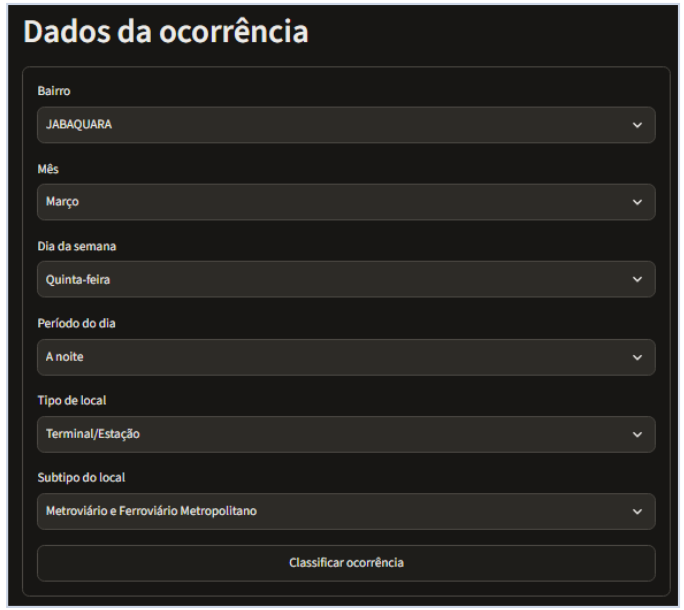
Não é o problema: treino ≈ 84% e validação ≈ 83%, e redes maiores não elevam esse teto. O limite vem da informação disponível (6 atributos de contexto), não da capacidade.

ERROS NO TESTE

20,5% dos roubos são classificados como furto (1.795) e 14,6% dos furtos como roubo (2.210). Útil como apoio experimental, não para decisões operacionais isoladas.

09 · APLICAÇÃO WEB

Aplicação web em Streamlit integrada ao modelo treinado





FUNCIONALIDADES

- Seleção de bairro, mês, dia da semana, período, tipo e subtipo de local
- Botão para classificar a ocorrência
- Exibe a classe estimada (roubo ou furto)
- Exibe as probabilidades de furto e de roubo e o limiar usado

Interpretação correta: "dado que a ocorrência tem estas características, o modelo atribui X% à classe roubo". Não é a chance de um roubo acontecer naquele local e horário.

Interface em Streamlit: **spmobilecrimeclassifier.streamlit.app/**

INTEGRAÇÃO COM A MLP

Formulário
6 atributos (Streamlit)

›

DataFrame
1 linha com os mesmos atributos

›

Pré-processador
one-hot do treino (joblib)

›

MLP
modelo .keras
293 → 64 → 32 → 1

›

Resultado
classe estimada + probabilidades

O formulário reflete exatamente os atributos do treino: as opções vêm das categorias aprendidas pelo codificador, e o mesmo pré-processamento é aplicado a cada consulta.

Resultados sólidos para um classificador experimental, com limites claros

CONCLUSÕES

- A MLP_1_Compacta separa roubo de furto com **83,2% de acurácia** e F1 macro de **82,1%** no teste.
- Três arquiteturas ficaram empatadas: com estes atributos, **mais capacidade não ajuda**; escolhemos a mais simples.
- Validação e teste coerentes indicam boa generalização.
- O modelo está integrado a uma aplicação web funcional.

LIMITAÇÕES

- Poucos atributos, todos categóricos: o desempenho se estabiliza em ≈ 83%.
- Dados sem padronização: grafias distintas de bairro; 48,6% sem período do dia.
- Só fatos registrados em boletim; apenas celulares, capital e 2025.
- Não estima risco de crime nem deve orientar policiamento.
- **Próximos passos:** normalizar bairros e testar atributos adicionais.

USO DE IA GENERATIVA

- Assistentes de IA apoiaram a **programação**, a **depuração**, a **documentação** e a análise dos resultados.
- A equipe **revisou e validou** o código e as decisões: dados, atributos, camadas, treinamento e escolha da arquitetura.
- A responsabilidade pela compreensão e pela validação da solução é da equipe.

Uso responsável: classificação do *tipo* de uma ocorrência já registrada, para fins acadêmicos e experimentais.